```
    hpen = CreatePen ( PS_DASHDOT, 1, RGB ( 255, 0, 0 ) ) ;
    holdpen = SelectObject ( hdc, hpen ) ;

    MoveToEx ( hdc, 10, 110, NULL ) ;
    LineTo ( hdc, 500, 110 ) ;

    SelectObject ( hdc, holdpen ) ;
    DeleteObject ( hpen ) ;

    hpen = CreatePen ( PS_DASHDOTDOT, 1, RGB ( 255, 0, 0 ) ) ;
    holdpen = SelectObject ( hdc, hpen ) ;

    MoveToEx ( hdc, 10, 160, NULL ) ;
    LineTo ( hdc, 500, 160 ) ;

    SelectObject ( hdc, holdpen ) ;
    DeleteObject ( hpen ) ;

    hpen = CreatePen ( PS_SOLID, 10, RGB ( 255, 0, 0 ) ) ;
    holdpen = SelectObject ( hdc, hpen ) ;

    MoveToEx ( hdc, 10, 210, NULL ) ;
    LineTo ( hdc, 500, 210 ) ;

    SelectObject ( hdc, holdpen ) ;
    DeleteObject ( hpen ) ;

    EndPaint ( hWnd, &ps ) ;
}
```

On execution of this program the window shown in Figure 18.3 appears.

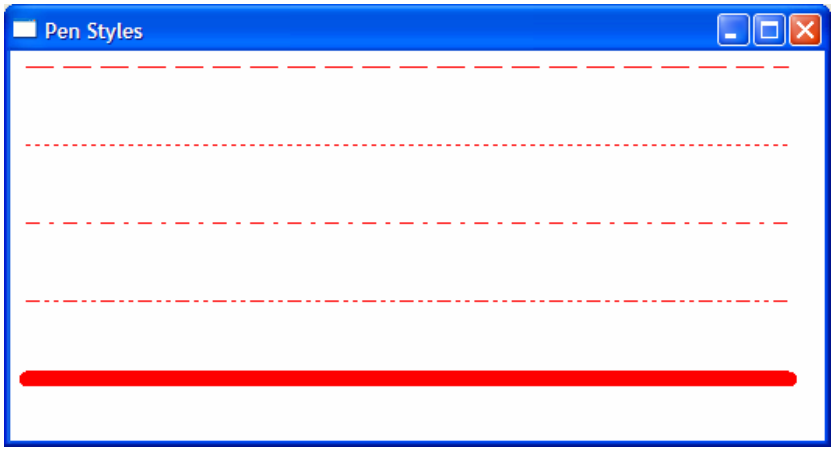

Figure 18.3

A new pen can be created using the **CreatePen( )** API function. This function needs three parameters—pen style, pen thickness and pen color. Different macros like PS_SOLID, PS_DOT, etc. have been defined in 'windows.h' to represent different pen styles. Note that for pen styles other than PS_SOLID the pen thickness has to be 1 pixel.

Types of Brushes

The way we can create different types of pens, we can also create three different types of brushes. These are—solid brush, hatch brush and pattern brush. Let us now write a program that shows how to build these brushes and then use them to fill rectangles. Here is the **OnPaint( )** handler which achieves this.

```
void OnPaint ( HWND hWnd )
{
    HDC hdc ;
    PAINTSTRUCT ps ;
    HBRUSH hbr ;
```

```
    HGDIOBJ holdbr ;
    HBITMAP hbmp ;

    hdc = BeginPaint ( hWnd, &ps ) ;

    hbr = CreateSolidBrush ( RGB (255, 0, 0 ) ) ;
    holdbr = SelectObject ( hdc, hbr ) ;

    Rectangle ( hdc, 5, 5, 105, 100 ) ;

    SelectObject ( hdc, holdbr ) ;
    DeleteObject ( hbr ) ;

    hbr = CreateHatchBrush ( HS_CROSS, RGB ( 255, 0, 0 ) ) ;
    holdbr = SelectObject ( hdc, hbr ) ;

    Rectangle ( hdc, 125, 5, 225, 100 ) ;

    SelectObject ( hdc, holdbr ) ;
    DeleteObject ( hbr ) ;

    hbmp = LoadBitmap ( hInst, MAKEINTRESOURCE ( IDB_BITMAP1 ) ) ;

    hbr = CreatePatternBrush ( hbmp ) ;
    holdbr = SelectObject ( hdc, hbr ) ;

    Rectangle ( hdc, 245, 5, 345, 100 ) ;

    SelectObject ( hdc, holdbr ) ;
    DeleteObject ( hbr ) ;
    DeleteObject ( hbmp ) ;

    EndPaint ( hWnd, &ps ) ;

    DeleteObject ( hbr ) ;
}
```

On execution of this program the window shown in Figure 18.4 appears.

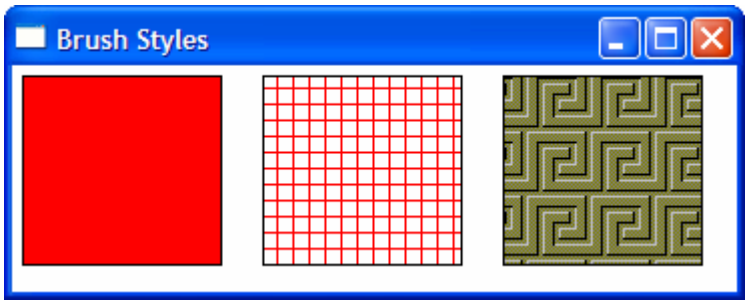

Figure 18.4

In the **OnPaint( )** handler we have drawn three rectangles—first using a solid brush, second using a hatched brush and third using a pattern brush. Creating and using a solid brush and hatched brush is simple. We simply have to make calls to **CreateSolidBrush( )** and **CreateHatchBrush( )** respectively. For the hatch brush we have used the style HS_CROSS. There are several other styles defined in 'windows.h' that you can experiment with.

For creating a pattern brush we need to first create a bitmap (pattern). Instead of creating this pattern, we have used a readymade bitmap file. You can use any other bitmap file present on your hard disk.

Bitmaps, menus, icons, cursors that a Windows program may use are its resources. When the compile such a program we usually want these resources to become a part of our EXE file. If so done we do not have to ship these resources separately. To be able to use a resource (bitmap file in our case) it is not enough to just copy it in the project directory. Instead we need to carry out the steps mentioned below to add a bitmap file to the project.

(a) From the 'Insert' menu option of VC++ 6.0 select the 'Resource' option.

(b) From the dialog that pops up select 'bitmap' followed by the import button.
(c) Select the suitable .bmp file.
(d) From the 'File' menu select the save option to save the generated resource script file (Script1.rc). When we select 'Save' one more file called 'resource.h' also gets created.
(e) Add the 'Script1.rc' file to the project using the Project | Add to Project | Files option.

While using the bitmap in the program it is always referred using an id. The id is **#define**d in the file 'resource.h'. Somewhere information has to be stored linking the id with the actual .bmp file on the disk. This is done in the 'Script1.rc' file. We need to include the 'resource.h' file in the program.

To create the pattern brush we first need to load the bitmap in memory. We have done this using the **LoadBitmap( )** API function. The first parameter passed to this function is the handle to the instance of the program. When **InitInstance( )** function is called from **WinMain( )** it stores the instance handle in a global variable **hInst**. We have passed this **hInst** to **LoadBitmap( )**. The second parameter passed to it is a string representing the bitmap. This string is created from the resource id using the **MAKEINTRESOURCE** macro. The **LoadBitmap( )** function returns the handle to the bitmap. This handle is then passed to the **CreatePatternBrush( )** function. This brush is then selected into the DC and then a rectangle is drawn using it.

Note that if the size of the bitmap is bigger than the rectangle being drawn then the bitmap is suitably clipped. On the other hand if the bitmap is smaller than the rectangle it is suitably replicated.

While doing the clean up firstly the brush is deleted followed by the bitmap.

Code and Resources

A program consists of both instructions and static data. Static data is that portion of the program which is not executed as machine instructions and which does not change as the program executes. Static data are character strings, data to create fonts, bitmaps, etc. The designers of Windows wisely decided that static data should be handled separately from the program code. The Windows term for static data is 'Resource data', or simply 'Resources'. By separating static data from the program code the creators of Windows were able to use a standard C/C++ compiler to create the code portion of the finished Windows program, and they only had to write a 'Resource compiler' to create the resources that Windows programs use. Separating the code from the resource data has other advantages like reducing memory demands and making programs more portable. It also means that a programmer can work on a program's logic, while a designer works on how the program looks.

Freehand Drawing, the Paintbrush Style

Even if you are knee high in computers I am sure you must have used PaintBrush. It provides a facility to draw a freehand drawing using mouse. Let us see if we too can achieve this. We can indicate where the freehand drawing begins by clicking the left mouse button. Then as we move the mouse on the table with the left mouse button depressed the freehand drawing should get drawn in the window. This drawing should continue till we do not release the left mouse button.

The mouse input comes in the form of messages. For free hand drawing we need to tackle three mouse messages—**WM_LBUTTONDOWN** for left button click, **WM_MOUSEMOVE** for mouse movement and **WM_LBUTTONUP** for releasing the left mouse button. Let us now see how these messages are tackled for drawing freehand. The

WndProc( ) function and the message handlers that perform this task are given below

```
int x1, y1, x2, y2 ;

LRESULT CALLBACK WndProc ( HWND hWnd, UINT message,
                                   WPARAM wParam, LPARAM lParam )
{
     switch ( message )
     {
          case WM_DESTROY :
               OnDestroy ( hWnd ) ;
               break ;

          case WM_LBUTTONDOWN :
               OnLButtonDown ( hWnd, LOWORD ( lParam ),
                                        HIWORD ( lParam ) ) ;
               break ;

          case WM_LBUTTONUP :
               OnLButtonUp( ) ;
               break ;

          case WM_MOUSEMOVE :
               OnMouseMove ( hWnd, wParam, LOWORD ( lParam ),
                                                HIWORD ( lParam ) ) ;
               break ;

          default:
               return DefWindowProc ( hWnd, message, wParam, lParam ) ;
  }
  return 0 ;
}

void OnLButtonDown ( HWND hWnd, int x, int y )
{
     SetCapture ( hWnd ) ;
     x1 = x ;
```

```
    y1 = y ;
}

void OnMouseMove ( HWND hWnd, int flags, int x, int y )
{
    HDC hdc ;
    if ( flags == MK_LBUTTON )  /* is left mouse button depressed */
    {
        hdc = GetDC ( hWnd ) ;
        x2 = x ;
        y2 = y ;
        MoveToEx ( hdc, x1, y1, NULL ) ;
        LineTo ( hdc, x2, y2 ) ;

        ReleaseDC ( hWnd, hdc ) ;

        x1 = x2 ;
        y1 = y2 ;
    }
}

void OnLButtonUp( )
{
    ReleaseCapture( ) ;
}
```

On execution of this program the window shown in Figure 18.5 appears. We can now click the left mouse button with mouse pointer placed anywhere in the window. We can then drag the mouse on the table to draw the freehand. The freehand drawing would continue till we do not release the left mouse button.

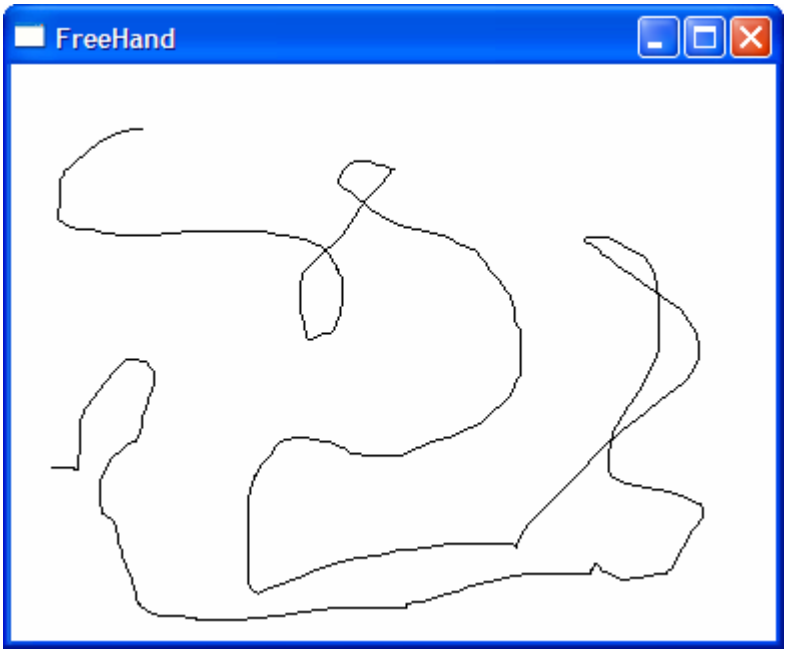

Figure 18.5

It appears that for drawing the freehand we should simply receive the mouse coordinates as it is moved and then highlight the pixels at these coordinates using the **SetPixel( )** API function. However, if we do so the freehand would be broken at several places. This is because usually the mouse is dragged pretty fast whereas the mouse move messages won't arrive so fast. A solution to this problem is to construct the freehand using small little line segments. This is what has been done in our program. These lines are so small is size that you would not even recognize that the freehand has been drawn by connecting these small lines.

Let us now discuss each mouse handler. When the **WM_LBUTTONDOWN** message arrives the **WndProc( )** function calls the handler **OnLButtonDown( )**. While doing so, we have passed the mouse coordinates where the click occurred. These coordinates are obtained in **lParam** in **WndProc( )**. In **lParam** the low order 16 bits contain the current x - coordinate of the mouse whereas the high order 16 bits contain the y - coordinate. The **LOWORD** and **HIWORD** macros have been used to separate out these x and y - coordinates from **lParam**.

In **OnLButtonDown( )** we have preserved the starting point of freehand in global variables **x1** and **y1**.

When **OnMouseMove( )** gets called it checks whether the left mouse button stands depressed. If it stands depressed then the **flags** variable contains **MK_LBUTTON**. If it does, then the current mouse coordinates are set up in the global variables **x2**, **y2**. A line is then drawn between **x1**, **y1** and **x2**, **y2** using the functions **MoveToEx( )** and **LineTo( )**. Next time around **x2**, **y2** should become the starting of the next line. Hence the current values of **x2**, **y2** are stored in **x1**, **y1**.

Note that here we have obtained the DC handle using the API function **GetDC( )**. This is because we are carrying out the drawing activity in reaction to a message other than **WM_PAINT**. Also, the handle obtained using **GetDC( )** should be released using a call to **ReleaseDC( )** function.

You can try using **BeginPaint( )** / **EndPaint( )** in mouse handlers and **GetDC( )** / **ReleaseDC( )** in **OnPaint( )**. Can you draw any conclusions?

Capturing the Mouse

If in the process of drawing the freehand the mouse cursor goes outside the client area then the window below our window would

start getting mouse messages. So our window would not receive any messages. If this has to be avoided then we should ensure that our window continues to receive mouse messages even when the cursor goes out of the client area of our window. The process of doing this is known as mouse capturing.

We have captured the mouse in **OnLButtonDown( )** handler by calling the API function **SetCapture( )**. As a result, the program continues to respond to mouse events during freehand drawing even if the mouse is moved outside the client area. In the **OnLButtonUp( )** handler we have released the captured mouse by calling the **ReleaseCapture( )** API function.

Device Context, a Closer Look

Now that we have written a few programs and are comfortable with idea of selecting objects like font, pen and brush into the DC, it is time for us to understand how Windows achieves the device independent drawing using the concept of DC. In fact a DC is nothing but a structure that holds handles of various drawing objects like font, pen, brush, etc. A screen DC and its working is shown in Figure 18.6.

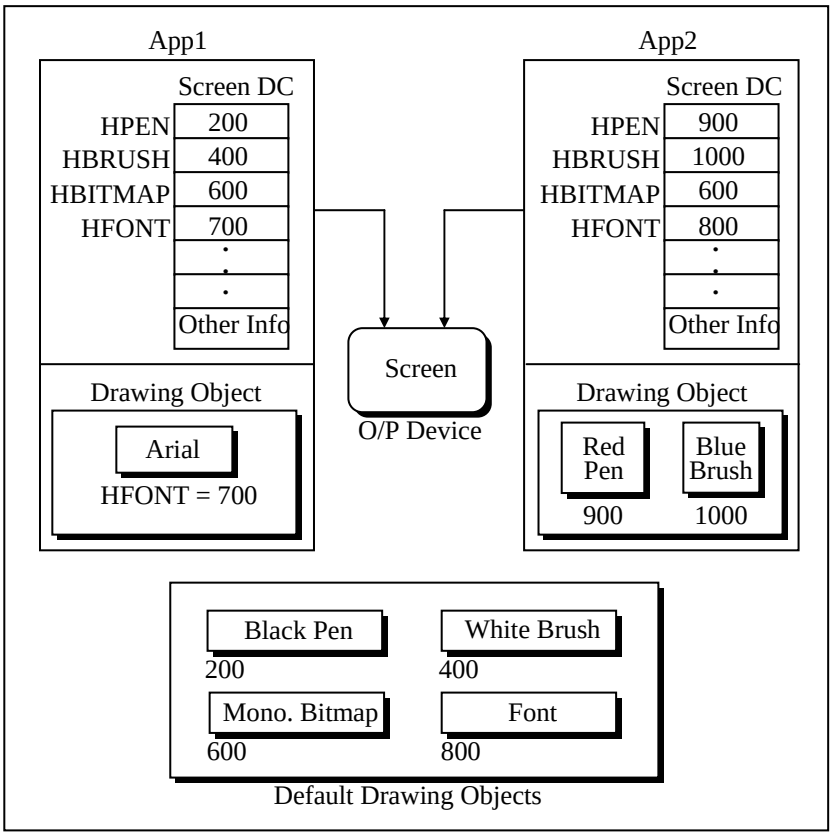

Figure 18.6

You can make following observations from Figure 18.6:

(a) The DC doesn't hold the drawing objects like pen, brush, etc. It merely holds their handles.
(b) With each DC a default monochrome bitmap of size 1 pixel x 1 pixel is associated.
(c) Default objects like black pen, white brush, etc. are shared by different DCs in same or different applications.

(d) The drawing objects that an application explicitly creates can be shared within DCs of the same application, but is never shared between different applications.
(e) Two different applications would need two different DCs even though both would be used to draw to the same screen. In other words with one screen multiple DCs can exist.
(f) A common Device Driver would serve the drawing requests coming from different applications. (Truly speaking the request comes from GDI functions that our application calls).

Screen and printer DC is OK, but what purpose would a memory DC serve? Well, that is what the next program would explain.

Displaying a Bitmap

We are familiar with drawing normal shapes on screen using a device context. How about drawing images on the screen? Windows does not permit displaying a bitmap image directly using a screen DC. This is because there might be color variations in the screen on which the bitmap was created and the screen on which it is being displayed. To account for such possibilities while displaying a bitmap Windows uses a different mechanism—a 'Memory DC'

The way anything drawn using a screen DC goes to screen, anything drawn using a printer DC goes to a printer, similarly anything drawn using a memory DC goes to memory (RAM). But where in RAM—in the 1 x 1 pixel bitmap whose handle is present in memory DC. (Note that this handle was of little use In case of screen/printer DC). Thus if we attempt to draw a line using a memory DC it would end up on the 1 x 1 pixel bitmap. You would agree 1 x 1 is too small a place to draw even a small line. Hence we need to expand the size and color capability of this bitmap. How can this be done? Simple, just replace the handle of the 1 x 1 bitmap with the handle of a bigger and colored bitmap object. This is shown in Figure 18.7.

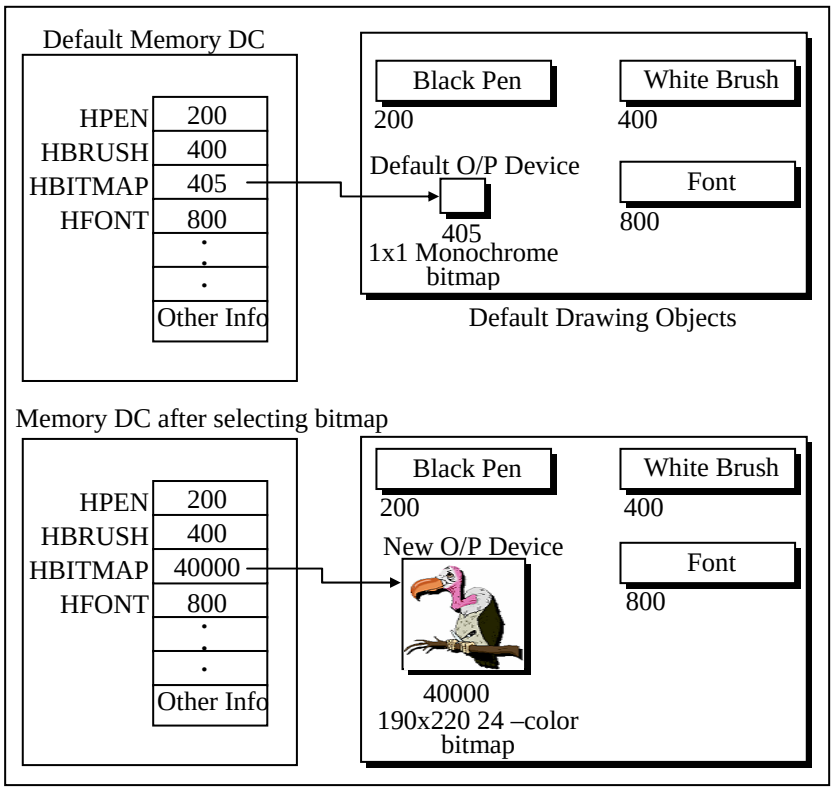

Figure 18.7

What purpose would just increasing the bitmap size/color would serve? Whatever we draw here would get drawn on the bitmap but would still not be visible. We can make it visible by simply copying the bitmap image (including what has been drawn on it) to the screen DC by using the API function **BitBlt( )**.

Before transferring the image to the screen DC we need to make the memory DC compatible with the screen DC. Here making compatible means making certain adjustments in the contents of the memory DC structure. Looking at these values the screen device driver would suitably adjust the colors when the pixels in

the bitmap of memory DC is transferred to screen DC using **BitBlt( )** function.

Let us now take a look at the program that puts all these concepts in action. The program merely displays the image of a vulture in a window. Here is the code…

```
void OnPaint ( HWND hWnd )
{
    HDC hdc ;
    HBITMAP hbmp ;
    HDC hmemdc ;
    HGDIOBJ holdbmp ;
    PAINTSTRUCT ps ;

    hdc = BeginPaint ( hWnd, &ps ) ;

    hbmp = LoadBitmap ( hInst, MAKEINTRESOURCE ( IDB_BITMAP1 ) ) ;

    hmemdc = CreateCompatibleDC ( hdc ) ;
    holdbmp = SelectObject ( hmemdc, hbmp ) ;

    BitBlt ( hdc, 10, 20, 190, 220, hmemdc, 0, 0, SRCCOPY ) ;

    EndPaint ( hWnd, &ps ) ;

    SelectObject ( hmemdc, holdbmp ) ;
    DeleteObject ( hbmp ) ;
    DeleteDC ( hmemdc ) ;
}
```

On executing the program we get the window shown in Figure 18.7.

Figure 18.7

As usual we begin our drawing activity in **OnPaint( )** by first getting the screen DC using the **BeginPaint( )** function. Next we have loaded the vulture bitmap image in memory by calling the **LoadBitmap( )** function. Its usage is similar to what we saw while creating a pattern brush in an earlier section of this chapter. Then we have created a memory device context and made its properties compatible with that of the screen DC. To do this we have called the API function **CreateCompatibleDC( )**. Note that we have passed the handle to the screen DC to this function. The function in turn returns the handle to the memory DC. After this we have selected the loaded bitmap into the memory DC. Lastly, we have performed a bit block transfer (a bit by bit copy) from memory DC to screen DC using the function **BitBlt( )**. As a result of this the vulture now appears in the window.

We have made the call to **BitBlt( )** as shown below:

```
BitBlt ( hdc, 10, 20, 190, 220, hmemdc, 0, 0, SRCCOPY ) ;
```

Let us now understand its parameters. These are as under:

hdc – Handle to target DC where the bitmap is to be blitted

10, 20 – Position where the bitmap is to be blitted

190, 220 – Width and height of bitmap being blitted

0, 0 – Top left corner of the source image. If we give 10, 20 then the image from 10, 20 to bottom right corner of the bitmap would get blitted.

SRCCOPY – Specifies one of the raster-operation codes. These codes define how the color data for the source rectangle is to be combined with the color data for the destination rectangle to achieve the final color. SRCCOPY means that the pixel color of source should be copied onto the destination pixel of the target.

Animation at Work

Speed is the essence of life. So having the ability to display a bitmap in a window is fine, but if we can add movement and sound to it then nothing like it. So let us now see how to achieve this animation and sound effect.

If we are to animate an object in the window we need to carry out the following steps:

(a) Create an image that is to be animated as a resource.
(b) Prepare the image for later display.
(c) Repeatedly display this prepared image at suitable places in the window taking care that when the next image is displayed the previous image is erased.
(d) Check for collisions while displaying the prepared image.

Let us now write a program that on execution makes a red colored ball move in the window. As the ball strikes the walls of the

window a noise occurs. Note that the width and height of the red-colored ball is 22 pixels. Given below is the **WndProc( )** function and the various message handlers that help achieve animation and sound effect.

```
HBITMAP hbmp ;
int x, y ;
HDC hmemdc ;
HGDIOBJ holdbmp ;

LRESULT CALLBACK WndProc ( HWND hWnd, UINT message,
                                    WPARAM wParam, LPARAM lParam )
{
    switch ( message )
    {
        case WM_DESTROY :
            OnDestroy ( hWnd ) ;
            break ;
        case WM_CREATE :
            OnCreate ( hWnd ) ;
            break ;
        case WM_TIMER :
            OnTimer ( hWnd ) ;
            break ;
        default :
            return DefWindowProc ( hWnd, message, wParam, lParam ) ;
    }
    return 0 ;
}

void OnCreate ( HWND hWnd )
{
    RECT r ;
    HDC hdc ;

    hbmp = LoadBitmap ( hInst, MAKEINTRESOURCE ( IDB_BITMAP1 ) ) ;

    hdc = GetDC ( hWnd ) ;
```